Experiment 'p_5_5_repeat_RERUN-1' Results

June 6, 2026

Experiment outcome: FAILURE_VERIFY
Bad responses: 1
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 3
Hard fails (resolution): 1
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_5_5_repeat_RERUN-1
Natural language statement: Write a method that returns a string repeated n times. For example, repeat("ho", 3) returns "hohoho".
Method signature: p_5_5_repeat(str: string, n: int) returns (repeated_str: string)

Ensures

- `|repeated_str| == n * |str|`
- `forall i :: 0 <= i < n ==> repeated_str[i * |str| .. (i + 1) * |str| ] == str`

Requires

- `n >= 0`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that returns a string repeated n times. For example, repeat("ho", 3)
    ↪ returns "hohoho".

The signature should be:

p_5_5_repeat(str: string, n: int) returns (repeated_str: string)

The method should respect the following contract:

requires n >= 0, ensures |repeated_str| == n * |str|, ensures forall i :: 0 <= i < n ==>
    ↪ repeated_str[i * |str| .. (i + 1) * |str| ] == str

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnjWthhYdAsS5EDOB1ZQQoHmbP1mx
Model: gpt-5.5-2026-04-23
Created at: 1780745459
Finish reason: stop
Usage: 378 tokens in, and 8044 tokens out

```
{
  repeated_str := "";
  var k := 0;
  var m := |str|;

  while k < n
    invariant 0 <= k <= n
    invariant m == |str|
    invariant 0 <= m
    invariant |repeated_str| == k * m
    invariant forall i :: 0 <= i < k ==> repeated_str[i * m .. (i + 1) * m] == str
    decreases n - k
  {
    var old_repeated_str := repeated_str;
    repeated_str := old_repeated_str + str;

    assert |repeated_str| == |old_repeated_str| + |str|;
    assert |old_repeated_str| == k * m;
    assert |str| == m;
```

```
  assert (k + 1) * m == k * m + m;
  assert |repeated_str| == (k + 1) * m;

  assert forall i | 0 <= i < k + 1 :: repeated_str[i * m .. (i + 1) * m] == str by {
    var a := i * m;
    var b := (i + 1) * m;
    assert 0 <= m;
    assert 0 <= a;
    assert a <= b;

    if i < k {
      assert old_repeated_str[a .. b] == str;
      assert i + 1 <= k;
      assert b <= k * m;
      assert b <= |old_repeated_str|;
      assert b <= |repeated_str|;

      assert |repeated_str[a .. b]| == |old_repeated_str[a .. b]|;
      assert forall j | 0 <= j < |repeated_str[a .. b]| :: repeated_str[a .. b][j] ==
          ↪ old_repeated_str[a .. b][j] by {
        assert 0 <= j < b - a;
        assert 0 <= a + j < b;
        assert a + j < |old_repeated_str|;
        assert repeated_str[a + j] == old_repeated_str[a + j];
        assert repeated_str[a .. b][j] == repeated_str[a + j];
        assert old_repeated_str[a .. b][j] == old_repeated_str[a + j];
      }
      assert repeated_str[a .. b] == old_repeated_str[a .. b];
    } else {
      assert i == k;
      assert a == k * m;
      assert a == |old_repeated_str|;
      assert b == (k + 1) * m;
      assert b == |old_repeated_str| + |str|;
      assert b == |repeated_str|;
      assert b - a == |str|;
      assert |repeated_str[a .. b]| == |str|;

      assert forall j | 0 <= j < |repeated_str[a .. b]| :: repeated_str[a .. b][j] ==
          ↪ str[j] by {
        assert 0 <= j < |str|;
        assert a == |old_repeated_str|;
        assert repeated_str[a + j] == str[j];
        assert repeated_str[a .. b][j] == repeated_str[a + j];
      }
      assert repeated_str[a .. b] == str;
    }

    assert repeated_str[a .. b] == str;
    assert repeated_str[i * m .. (i + 1) * m] == str;
  }
```

```
    k := k + 1;
  }

  assert k == n;
  assert |repeated_str| == n * |str|;
  assert forall i | 0 <= i < n :: repeated_str[i * |str| .. (i + 1) * |str|] == str by {
    assert 0 <= i < k;
    assert m == |str|;
    assert i * m == i * |str|;
    assert (i + 1) * m == (i + 1) * |str|;
    assert repeated_str[i * m .. (i + 1) * m] == str;
  }
}
```

Program → GenAI

```
When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_5_5_repeat_RERUN-1/program.dfy(28,15): Error: unresolved identifier: i
   |
28 | var a := i * m;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(29,16): Error: unresolved identifier: i
   |
29 | var b := (i + 1) * m;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(34,9): Error: unresolved identifier: i
   |
34 | if i < k {
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(36,15): Error: unresolved identifier: i
   |
36 | assert i + 1 <= k;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(43,22): Error: unresolved identifier: j
   |
43 | assert 0 <= j < b - a;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(44,26): Error: unresolved identifier: j
   |
44 | assert 0 <= a + j < b;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(45,21): Error: unresolved identifier: j
   |
```

```
45 | assert a + j < |old_repeated_str|;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(46,34): Error: unresolved identifier: j
   |
46 | assert repeated_str[a + j] == old_repeated_str[a + j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(46,61): Error: unresolved identifier: j
   |
46 | assert repeated_str[a + j] == old_repeated_str[a + j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(47,38): Error: unresolved identifier: j
   |
47 | assert repeated_str[a .. b][j] == repeated_str[a + j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(47,61): Error: unresolved identifier: j
   |
47 | assert repeated_str[a .. b][j] == repeated_str[a + j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(48,42): Error: unresolved identifier: j
   |
48 | assert old_repeated_str[a .. b][j] == old_repeated_str[a + j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(48,69): Error: unresolved identifier: j
   |
48 | assert old_repeated_str[a .. b][j] == old_repeated_str[a + j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(52,15): Error: unresolved identifier: i
   |
52 | assert i == k;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(62,22): Error: unresolved identifier: j
   |
62 | assert 0 <= j < |str|;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(64,34): Error: unresolved identifier: j
   |
64 | assert repeated_str[a + j] == str[j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(64,44): Error: unresolved identifier: j
   |
64 | assert repeated_str[a + j] == str[j];
```

```
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(65,38): Error: unresolved identifier: j
   |
65 | assert repeated_str[a .. b][j] == repeated_str[a + j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(65,61): Error: unresolved identifier: j
   |
65 | assert repeated_str[a .. b][j] == repeated_str[a + j];
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(71,26): Error: unresolved identifier: i
   |
71 | assert repeated_str[i * m .. (i + 1) * m] == str;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(71,36): Error: unresolved identifier: i
   |
71 | assert repeated_str[i * m .. (i + 1) * m] == str;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(80,16): Error: unresolved identifier: i
   |
80 | assert 0 <= i < k;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(82,11): Error: unresolved identifier: i
   |
82 | assert i * m == i * |str|;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(82,20): Error: unresolved identifier: i
   |
82 | assert i * m == i * |str|;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(83,12): Error: unresolved identifier: i
   |
83 | assert (i + 1) * m == (i + 1) * |str|;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(83,27): Error: unresolved identifier: i
   |
83 | assert (i + 1) * m == (i + 1) * |str|;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(84,24): Error: unresolved identifier: i
   |
84 | assert repeated_str[i * m .. (i + 1) * m] == str;
   | ^
```

```
problem_data/p_5_5_repeat_RERUN-1/program.dfy(84,34): Error: unresolved identifier: i
   |
84 | assert repeated_str[i * m .. (i + 1) * m] == str;
   |                                  ^

28 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnjZSa2ESS2LzBg24QDdQGhmGswHH
Model: gpt-5.5-2026-04-23
Created at: 1780745618
Finish reason: stop
Usage: 4212 tokens in, and 1478 tokens out

```
{
  repeated_str := "";
  var k := 0;

  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
    decreases n - k
  {
    var old_repeated_str := repeated_str;
    repeated_str := old_repeated_str + str;

    assert |repeated_str| == |old_repeated_str| + |str|;
    assert |old_repeated_str| == k * |str|;
    assert |repeated_str| == (k + 1) * |str|;

    forall i | 0 <= i < k + 1
      ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
    {
      if i < k {
        assert old_repeated_str[i * |str| .. (i + 1) * |str|] == str;
        assert (i + 1) * |str| <= k * |str|;
        assert (i + 1) * |str| <= |old_repeated_str|;
        assert repeated_str[i * |str| .. (i + 1) * |str|] == old_repeated_str[i * |str| ..
            ↪  (i + 1) * |str|];
      } else {
        assert i == k;
        assert i * |str| == |old_repeated_str|;
        assert (i + 1) * |str| == |old_repeated_str| + |str|;
```

```
      assert repeated_str[i * |str| .. (i + 1) * |str|] == str;
    }
   }

   k := k + 1;
  }
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_5_5_repeat_RERUN-1/program.dfy(26,31): Error: upper bound below lower
    ↪ bound or above length of sequence
   |
26 | assert old_repeated_str[i * |str| .. (i + 1) * |str|] == str;
   | ^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(27,8): Error: assertion might not hold
   |
27 | assert (i + 1) * |str| <= k * |str|;
   | ^^^^^^

problem_data/p_5_5_repeat_RERUN-1/program.dfy(29,8): Error: assertion might not hold
   |
29 | assert repeated_str[i * |str| .. (i + 1) * |str|] == old_repeated_str[i * |str| .. (
    ↪ i + 1) * |str|];
   | ^^^^^^


Dafny program verifier finished with 1 verified, 3 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnjaEmKSGDd4GyggP00EYVmXMhwlP
Model: gpt-5.5-2026-04-23
Created at: 1780745666
Finish reason: stop
Usage: 3721 tokens in, and 7449 tokens out

```
{
  repeated_str := "";
  var k := 0;
  var m := |str|;

  while k < n
```

```
  invariant 0 <= k <= n
  invariant m == |str|
  invariant 0 <= m
  invariant |repeated_str| == k * m
  invariant |repeated_str| == k * |str|
  invariant forall i:int :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] ==
      ↪ str
  decreases n - k
{
  var old_repeated_str := repeated_str;
  assert |old_repeated_str| == k * m;
  assert forall i:int :: 0 <= i < k ==> old_repeated_str[i * |str| .. (i + 1) * |str|]
      ↪ == str;

  repeated_str := old_repeated_str + str;

  assert |repeated_str| == |old_repeated_str| + |str|;
  assert |old_repeated_str| == k * m;
  assert |str| == m;
  assert |repeated_str| == k * m + m;
  assert k * m + m == (k + 1) * m;
  assert |repeated_str| == (k + 1) * m;
  assert |repeated_str| == (k + 1) * |str|;

  forall i:int | 0 <= i < k + 1
    ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var a := i * m;
    var b := (i + 1) * m;
    assert a == i * |str|;
    assert b == (i + 1) * |str|;
    assert 0 <= i;
    assert 0 <= m;
    assert 0 <= a;
    assert b == a + m;
    assert a <= b;

    if i < k {
      assert i + 1 <= k;
      var d := k - (i + 1);
      assert 0 <= d;
      assert k == i + 1 + d;
      assert k * m == (i + 1 + d) * m;
      assert (i + 1 + d) * m == (i + 1) * m + d * m;
      assert 0 <= d * m;
      assert (i + 1) * m <= (i + 1) * m + d * m;
      assert (i + 1) * m <= k * m;
      assert b <= k * m;
      assert b <= |old_repeated_str|;
      assert b <= |repeated_str|;
```

```
      assert old_repeated_str[a .. b] == str;

      forall j:int | 0 <= j < b - a
        ensures repeated_str[a .. b][j] == old_repeated_str[a .. b][j]
      {
        assert 0 <= a + j;
        assert a + j < b;
        assert a + j < |old_repeated_str|;
        assert repeated_str == old_repeated_str + str;
        assert repeated_str[a + j] == old_repeated_str[a + j];
        assert repeated_str[a .. b][j] == repeated_str[a + j];
        assert old_repeated_str[a .. b][j] == old_repeated_str[a + j];
      }
      assert |repeated_str[a .. b]| == |old_repeated_str[a .. b]|;
      assert repeated_str[a .. b] == old_repeated_str[a .. b];
      assert repeated_str[a .. b] == str;
    } else {
      assert i == k;
      assert a == k * m;
      assert a == |old_repeated_str|;
      assert b == (k + 1) * m;
      assert b == k * m + m;
      assert b == |old_repeated_str| + |str|;
      assert b == |repeated_str|;
      assert b - a == |str|;

      forall j:int | 0 <= j < |str|
        ensures repeated_str[a .. b][j] == str[j]
      {
        assert 0 <= j < b - a;
        assert repeated_str[a .. b][j] == repeated_str[a + j];
        assert a == |old_repeated_str|;
        assert a + j == |old_repeated_str| + j;
        assert repeated_str == old_repeated_str + str;
        assert repeated_str[a + j] == str[j];
      }
      assert |repeated_str[a .. b]| == |str|;
      assert repeated_str[a .. b] == str;
    }

    assert repeated_str[a .. b] == str;
    assert repeated_str[i * |str| .. (i + 1) * |str|] == str;
  }

  k := k + 1;
 }

 assert k == n;
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_5_5_repeat_RERUN-1/program.dfy(1,7): Error: Verification of 'p_5_5_repeat'
    ↪  timed out after 30 seconds. (the limit can be increased using --verification-time
    ↪ -limit)
  |
1 | method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
  | ^^^^^^^^^^^^


Dafny program verifier finished with 1 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnjdBYd1KzryEZWTIC5PYrz1dPtoE
Model: gpt-5.5-2026-04-23
Created at: 1780745849
Finish reason: length
Usage: 4961 tokens in, and 4718 tokens out
The below is a raw response from GenAI that could not be parsed. It is likely that GenAI got 'stuck' in a loop and hit the pre-set token limit (i.e., finish reason is 'length').

Final Program

```
method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
       requires n >= 0
       ensures |repeated_str| == n * |str|
       ensures forall i :: 0 <= i < n ==> repeated_str[i * |str| .. (i + 1) * |str| ] ==
           ↪ str
{
  repeated_str := "";
  var k := 0;
  var m := |str|;

  while k < n
    invariant 0 <= k <= n
    invariant m == |str|
    invariant 0 <= m
    invariant |repeated_str| == k * m
    invariant |repeated_str| == k * |str|
    invariant forall i:int :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] ==
        ↪  str
```

```
  decreases n - k
{
  var old_repeated_str := repeated_str;
  assert |old_repeated_str| == k * m;
  assert forall i:int :: 0 <= i < k ==> old_repeated_str[i * |str| .. (i + 1) * |str|]
      ↪ == str;

  repeated_str := old_repeated_str + str;

  assert |repeated_str| == |old_repeated_str| + |str|;
  assert |old_repeated_str| == k * m;
  assert |str| == m;
  assert |repeated_str| == k * m + m;
  assert k * m + m == (k + 1) * m;
  assert |repeated_str| == (k + 1) * m;
  assert |repeated_str| == (k + 1) * |str|;

  forall i:int | 0 <= i < k + 1
    ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var a := i * m;
    var b := (i + 1) * m;
    assert a == i * |str|;
    assert b == (i + 1) * |str|;
    assert 0 <= i;
    assert 0 <= m;
    assert 0 <= a;
    assert b == a + m;
    assert a <= b;

    if i < k {
      assert i + 1 <= k;
      var d := k - (i + 1);
      assert 0 <= d;
      assert k == i + 1 + d;
      assert k * m == (i + 1 + d) * m;
      assert (i + 1 + d) * m == (i + 1) * m + d * m;
      assert 0 <= d * m;
      assert (i + 1) * m <= (i + 1) * m + d * m;
      assert (i + 1) * m <= k * m;
      assert b <= k * m;
      assert b <= |old_repeated_str|;
      assert b <= |repeated_str|;

      assert old_repeated_str[a .. b] == str;

      forall j:int | 0 <= j < b - a
        ensures repeated_str[a .. b][j] == old_repeated_str[a .. b][j]
      {
        assert 0 <= a + j;
        assert a + j < b;
```

```
        assert a + j < |old_repeated_str|;
        assert repeated_str == old_repeated_str + str;
        assert repeated_str[a + j] == old_repeated_str[a + j];
        assert repeated_str[a .. b][j] == repeated_str[a + j];
        assert old_repeated_str[a .. b][j] == old_repeated_str[a + j];
      }
      assert |repeated_str[a .. b]| == |old_repeated_str[a .. b]|;
      assert repeated_str[a .. b] == old_repeated_str[a .. b];
      assert repeated_str[a .. b] == str;
    } else {
      assert i == k;
      assert a == k * m;
      assert a == |old_repeated_str|;
      assert b == (k + 1) * m;
      assert b == k * m + m;
      assert b == |old_repeated_str| + |str|;
      assert b == |repeated_str|;
      assert b - a == |str|;

      forall j:int | 0 <= j < |str|
        ensures repeated_str[a .. b][j] == str[j]
      {
        assert 0 <= j < b - a;
        assert repeated_str[a .. b][j] == repeated_str[a + j];
        assert a == |old_repeated_str|;
        assert a + j == |old_repeated_str| + j;
        assert repeated_str == old_repeated_str + str;
        assert repeated_str[a + j] == str[j];
      }
      assert |repeated_str[a .. b]| == |str|;
      assert repeated_str[a .. b] == str;
    }

    assert repeated_str[a .. b] == str;
    assert repeated_str[i * |str| .. (i + 1) * |str|] == str;
  }

  k := k + 1;
 }

 assert k == n;
}
```

Total Token Usage

Input tokens: 13272
Output tokens: 21689
Reasoning tokens: 18946
Sum of 'total tokens': 34961

Experiment Timings

Overall Experiment started at 1780745457946, ended at 1780745947642, lasting 489696ms (489.70 seconds)
Iteration #4 started at 1780745848373, ended at 1780745947642, lasting 99269ms (99.27 seconds)
Iteration #1 started at 1780745457947, ended at 1780745617795, lasting 159848ms (159.85 seconds)
Iteration #2 started at 1780745617795, ended at 1780745665813, lasting 48018ms (48.02 seconds)
Iteration #3 started at 1780745665813, ended at 1780745848371, lasting 182558ms (182.56 seconds)